Computational Vision

Diego Chiola

1 Pixel Basics, Image Types, and Color

1.1 Image Types and Pixel Contents

When we think of digital images, we usually think of standard photographs. However, in computational vision, the value stored inside a pixel depends entirely on what kind of sensor captured the image:

- **Pictorial Digital Images (Photos):** These capture light intensity (yielding black-and-white images) or color.
- **Range Images:** Instead of light, the pixels store **depth information** (how far away an object is from the sensor). This is exactly how tools like Microsoft Kinect or Apple's FaceID operate.
- **Medical Images:** Pixels represent the radiation absorbance level of different tissues (e.g., X-rays or CT scans).
- **Thermal Images:** Pixels capture heat signatures, allowing vision in complete darkness.

1.2 Dynamic Range and Quantization

The **dynamic range** of an image refers to the total number of distinctive values that can possibly occur in the image. Think of this as the "palette" of available shades.

This range is limited by two things: the physical capabilities of the camera sensor and the number of bits (data) we choose to assign to each pixel, a process called quantization.

- **1-bit depth:** The simplest image. A pixel can only be 0 or 1, meaning the image is strictly black and white with no shades of gray.
- **8-bit depth (Gray Level):** This is the standard for grayscale images. We associate 1 byte (which is 8 bits) to each pixel.

Numerical Example: Because computers run on binary, an 8-bit depth means we have 2^8 possible combinations.

$$2^8 = 256 \text{ gray levels.}$$

This means a pixel can hold any integer value from 0 (pure black) to 255 (pure white). A value of 128 would be a perfect mid-tone gray.

1.3 Color Image Processing

To capture color, cameras use three different physical filters (sensitive to Red, Green, and Blue light) to acquire three separate monochrome images. These three layers are then stacked together to form a single RGB image.

- **Full Color (24-bit):** In standard color images, every single pixel is described by a triplet of values: (R, G, B).
- Because each of the three channels gets its own 8-bit byte (256 possible values), a single color pixel requires 3 bytes of memory.

Numerical Example: Let's look at the math behind "millions of colors". If Red, Green, and Blue each have 256 possible levels, the total number of color combinations is:

$$256 \times 256 \times 256 = 16,777,216 \text{ possible colors.}$$

For instance, a pixel with the triplet $(255, 0, 0)$ is telling the monitor to display maximum Red, zero Green, and zero Blue. A triplet of $(255, 255, 255)$ mixes all light to create pure white.

When processing these images algorithmically, engineers typically face a choice: either process the R, G, and B channels completely independently and merge the results at the end, or devise ad-hoc algorithms that analyze the complex interplay of the colors together.

2 Global Descriptors and Image Segmentation

2.1 The Goal: Image Segmentation

One of the most fundamental tasks in computational vision is **image segmentation**. This means dividing an image into meaningful parts, essentially detecting groups of pixels that are physically close together and visually similar.

We can evaluate "similarity" based on various attributes, such as:

- **Brightness** (in grayscale images)
- **Color**
- **Texture** or **Motion**

The simplest example is separating a foreground object (like the cameraman in the slides) from the background.

2.2 Brightness Histograms and Binarization

To figure out which pixels belong to the foreground and which belong to the background, we use a **brightness histogram**.

- **What is it?** A histogram is a graph where the x-axis represents the possible pixel values (0 to 255 for an 8-bit image) and the y-axis represents the number of pixels in the image that have that exact value.

If an image has a dark background and a bright subject, the histogram will typically show two distinct "peaks" (a bimodal histogram). We can achieve segmentation through **binarization**. This involves picking a threshold value directly between those two peaks.

Numerical Example: Imagine an image where the background pixels mostly have values around 30 (dark gray) and the object pixels are around 200 (light gray). We can set a threshold at $T = 100$. The algorithm looks at every pixel $I(x, y)$:

- If $I(x, y) < 100$, it sets the pixel to 0 (pure black).
- If $I(x, y) \geq 100$, it sets the pixel to 255 (pure white).

The result is a clean binary image (a silhouette).

2.3 The Challenge of Contrast

The slides point out a critical reality: finding that perfect threshold is rarely easy! The shape of the histogram tells us a lot about the image quality:

- **Dark image:** All pixels are bunched up on the left side (near 0).
- **Bright image:** All pixels are bunched up on the right side (near 255).
- **Low-contrast image:** The pixels are squished together in a narrow spike in the middle. It's very hard to separate objects here.
- **High-contrast image:** The pixels are spread widely across the entire 0-255 range. This is ideal for thresholding.

2.4 Dealing with Color

When we move to color images, segmentation gets trickier. The slides note that in color, values being numerically "close" in raw RGB code doesn't always mean they look visually "similar" to human eyes.

For example, a pixel in shadow might have entirely different RGB numbers than a sunlit pixel of the exact same object. Because of this, a careful choice of the **color space** (like moving from RGB to HSV - Hue, Saturation, Value) is highly recommended before trying to segment.

2.5 Image Quantization

Another technique mentioned is **Image Quantization**, which is the deliberate reduction of the image's dynamic range. Instead of having 256 shades of gray, we might force the image to only use 4 or 8 shades. This simplifies the image drastically, washing out minor noise and making it a very useful pre-processing step before attempting segmentation.

Numerical Example: Let's say we want to reduce 256 levels down to just 4 levels. We divide the range into 4 "buckets" of 64 values each. For a pixel with an original value of 150:

$$New\_Value = \left\lfloor \frac{150}{64} \right\rfloor = 2$$

We map it to bucket "2", completely erasing the subtle differences between value 150 and value 160 (they both go into the same bucket).

2.6 Summary of Global Descriptors (Histograms)

Histograms are used to obtain an overall description of the image content.

- **The Good:** They are very easy to compute and highly robust to geometrical variations. If you take a picture of an apple and rotate the photo 90 degrees, the histogram remains exactly the same.
- **The Bad:** They completely **lose spatial information**.

Context: To understand why losing spatial information is bad, imagine an image of a chessboard. Now imagine a second image where the top half is solid black and the bottom half is solid white. Both images have 50% black pixels and 50% white pixels. Their histograms will be absolutely identical, even though the images look completely different!

3 Linear Filters and Local Features

3.1 Linear Filtering and Convolution

A basic tool in image processing is the linear filter, used for noise reduction and signal enhancement. Filtering is achieved through a mathematical operation called **convolution**.

In convolution, a small matrix called a **kernel** (k) is swept across the image signal (f) to produce a new filtered image (g):

$$g = f * k$$

For a 2D image, the math is the sum of the element-wise products between the kernel and the image patch it currently covers:

$$g[x, y] = \sum_{m,l} f[x - m, y - l]k[m, l]$$

Numerical Example: Imagine a 1D row of pixels transitioning from dark to light: $f = [10, 10, 10, 50, 50, 50]$. We want to find the edge using a simple edge-detection kernel: $k = [-1, 0, 1]$. Let's center the kernel over the third pixel (value 10):

$$g[3] = (10 \times -1) + (10 \times 0) + (50 \times 1) = -10 + 0 + 50 = 40$$

The high output value (40) indicates that a sharp transition (an edge) was detected at this location.

3.2 Image Gradients and Edges

Edges are pixels at which the image values undergo a sharp variation. To find them, we compute the **Image Gradient**, a vector that points in the direction of the most rapid change in intensity.

The gradient is composed of partial derivatives in the x and y directions:

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

From this, we extract two crucial pieces of geometric information:

- **Gradient Magnitude** (Edge strength): $M(x, y) = \sqrt{g_x^2 + g_y^2}$
- **Gradient Orientation** (Edge direction): $\theta(x, y) = \arctan\left(\frac{g_y}{g_x}\right)$

3.3 Image Gradient: Numerical Example

To calculate the gradient at a specific pixel, we estimate the partial derivatives using the **central difference** of its immediate neighbors.

Imagine a 3×3 patch of grayscale image pixels, where we want to find the gradient of the center pixel (x, y):

$$\text{Patch} = \begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{50} & 90 \\ 10 & 90 & 170 \end{bmatrix}$$

3.3.1 Calculate Partial Derivatives

We find the rate of change in the horizontal (x) and vertical (y) directions. *(Note: In standard image coordinates, the y-axis increases downwards).*

- **Horizontal change (g_x):** Pixel to the Right - Pixel to the Left

$$g_x = f(x+1, y) - f(x-1, y) = 90 - 10 = 80$$

- **Vertical change (g_y):** Pixel Below - Pixel Above

$$g_y = f(x, y+1) - f(x, y-1) = 90 - 10 = 80$$

3.3.2 Calculate Gradient Magnitude

The magnitude measures the **strength** of the edge.

$$M(x, y) = \sqrt{g_x^2 + g_y^2}$$

$$M = \sqrt{80^2 + 80^2} = \sqrt{6400 + 6400} = \sqrt{12800} \approx 113.1$$

3.3.3 Calculate Gradient Orientation

The orientation measures the **direction** of the most rapid increase in intensity.

$$\theta(x, y) = \arctan\left(\frac{g_y}{g_x}\right)$$

$$\theta = \arctan\left(\frac{80}{80}\right) = \arctan(1) = 45^\circ$$

Conclusion: The gradient vector points at a 45° angle (down and to the right). The gradient always points in the direction of steepest ascent (from dark to bright).

3.4 The Effect of Noise and Gaussian Smoothing

Calculating derivatives is highly sensitive to noise. Taking the derivative of a noisy signal results in massive spikes that hide the true edges.

The Solution: We must smooth the image before taking the derivative. We typically use a **Gaussian Filter**, which acts as a low-pass filter to blur out random noise.

$$h_\sigma(u,v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

Mathematical Shortcut: Because convolution is associative, taking the derivative of a smoothed image is mathematically identical to smoothing the image with the *derivative of the smoothing filter*:

$$\frac{\partial}{\partial x}(h * f) = \left(\frac{\partial}{\partial x} h\right) * f$$

This saves computational power by allowing us to use a single "Derivative of Gaussian" filter. This logic forms the foundation of the famous **Canny Edge Detector**.

3.5 Local Features: Corners

While edges are useful, they suffer from the aperture problem (you cannot tell where along an edge you are). **Corners** are patches with strong gradients in at least *two* significantly different orientations, making them stable and "good features to match" across different images.

3.5.1 The Autocorrelation Function

To detect corners, we analyze how a local patch of pixels changes when shifted by a small vector u. We use the Summed Square Difference (SSD), also known as the autocorrelation function $E_{AC}(u)$:

$$E_{AC}(u) = \sum_i [I(x_i + u) - I(x_i)]^2$$

To make this computationally feasible, we approximate the shifted image using a Taylor series expansion:

$$I(x_i + u) \approx I(x_i) + \nabla I(x_i) \cdot u$$

Substituting this back into the SSD equation cancels out the $I(x_i)$ terms, yielding a quadratic form:

$$E_{AC}(u) \approx \sum_i [\nabla I(x_i) \cdot u]^2 = u^\top A u$$

3.5.2 The Autocorrelation Matrix (A)

The matrix A captures the gradient distribution within the patch:

$$A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

By calculating the eigenvalues (λ_1, λ_2) of A, we can classify the local geometry:

- **Corner:** λ_1 and λ_2 are both large.
- **Edge:** $\lambda_1 \gg \lambda_2$ or $\lambda_2 \gg \lambda_1$.
- **Flat Region:** Both λ_1 and λ_2 are small.

3.5.3 Corner Numerical Example

Consider a 4×4 image grid containing a dark square (intensity 10) on a light background (intensity 100). We will compute the Autocorrelation Matrix for the inner 2×2 patch.

$$\text{Image} = \begin{bmatrix} 100 & 100 & 100 & 100 \\ 100 & \mathbf{10} & \mathbf{10} & 100 \\ 100 & \mathbf{10} & \mathbf{10} & 100 \\ 100 & 100 & 100 & 100 \end{bmatrix}$$

Step 1: Calculate Central Difference Gradients (I_x, I_y) For the four inner pixels:

- **Top-Left:** $I_x = 10 - 100 = -90$, $I_y = 10 - 100 = -90$
- **Top-Right:** $I_x = 100 - 10 = 90$, $I_y = 10 - 100 = -90$
- **Bottom-Left:** $I_x = 10 - 100 = -90$, $I_y = 100 - 10 = 90$
- **Bottom-Right:** $I_x = 100 - 10 = 90$, $I_y = 100 - 10 = 90$

Step 2: Calculate Matrix Components Squaring and summing these values over the 4 pixels:

$$\sum I_x^2 = (-90)^2 + (90)^2 + (-90)^2 + (90)^2 = 32400$$

$$\sum I_y^2 = (-90)^2 + (-90)^2 + (90)^2 + (90)^2 = 32400$$

$$\sum I_x I_y = 8100 - 8100 - 8100 + 8100 = 0$$

Step 3: Construct Matrix A and Extract Eigenvalues

$$A = \begin{bmatrix} 32400 & 0 \\ 0 & 32400 \end{bmatrix}$$

Because A is a diagonal matrix, its eigenvalues are simply the diagonal entries:

$$\lambda_1 = 32400$$

$$\lambda_2 = 32400$$

Conclusion: Because both eigenvalues are exceedingly large, the algorithm confidently classifies this patch as a **Corner**.

4 Image Segmentation

Image segmentation is a fundamental task in computational vision that involves partitioning a digital image into multiple segments (sets of pixels, also known as image objects). The goal of segmentation is to simplify and/or change the representation of an image into something that is more meaningful and easier to analyze.

4.1 Core Concept: Uniformity

Segmentation typically divides an image into regions that are uniform according to a specific quality or feature:

- **Brightness/Intensity:** Common light levels.
- **Color:** Similarities in the color space.
- **Motion:** Pixels moving with the same velocity/direction.

4.2 Mathematical Definition

Let R represent the entire spatial region occupied by an image. Image segmentation is the process of partitioning R into n subregions $R_1, R_2, \ldots, R_n$ such that:

1. **Completeness:** $\bigcup_{i=1}^{n} R_i = R$. Every pixel must belong to a region.
2. **Connectivity:** R_i is a connected set for $i = 1, \ldots, n$.
3. **Disjointness:** $R_i \cap R_j = \emptyset$ for all $i \neq j$. Regions cannot overlap.
4. **Uniformity Predicate:** $Q(R_i) = \text{TRUE}$ for $i = 1, \ldots, n$. All pixels in a region satisfy a logical predicate Q (e.g., same intensity).
5. **Maximality:** $Q(R_i \cup R_j) = \text{FALSE}$ for any adjacent regions R_i and R_j. Neighboring regions must be different according to the predicate Q.

4.3 Methods Overview

Segmentation methods can be broadly categorized based on the level of guidance and the visual cues used.

4.3.1 Supervision Levels

- **Unsupervised:** Segmentation is based purely on pixel and appearance information (low-level features).
- **Supervised:** Segmentation is guided by semantic concepts or prior knowledge of specific object classes.

4.3.2 Technical Approaches

- **Edge-based methods:** Look for discontinuities (edges) in the image to find boundaries between regions.
- **Region-based methods:** Look for regions of similarity according to specific criteria. This is generally considered a more robust formulation.

5 Thresholding and Motion Segmentation

This chapter focuses on **Thresholding**, a region-based segmentation method that groups pixels sharing similar intensity levels.

5.1 Basic Thresholding Concepts

The simplest form of segmentation is the binary case, where we separate an image into two classes: Foreground and Background. This is achieved by comparing each pixel intensity $f(x, y)$ to a global threshold T.

5.1.1 Global Binary Thresholding

The resulting image $g(x, y)$ is defined as:

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

Where 1 typically represents the foreground and 0 the background.

5.1.2 Multiple Thresholding

When an image contains multiple objects with distinct intensity ranges, a single threshold is insufficient. Multiple thresholds $(T_1, T_2, \dots)$ can be used:

$$g(x,y) = \begin{cases} a & \text{if } f(x,y) > T_2 \\ b & \text{if } T_1 < f(x,y) \leq T_2 \\ c & \text{otherwise} \end{cases} \tag{2}$$

5.2 Motion Segmentation

Motion segmentation is a practical application of thresholding where the "uniformity" criterion is motion rather than static intensity. By identifying pixels that change significantly between frames, we can extract moving objects from a static background. This is a binary segmentation task (Moving vs. Not Moving).

5.3 Automatic Global Threshold Selection

Instead of manually picking T, we can use an iterative algorithm to find the optimal value, typically located in the "valley" of a bimodal histogram.

5.3.1 The Iterative Algorithm

1. **Initial Estimate:** Select an initial T (often the global mean intensity of the image).
2. **Segmentation:** Partition the image into two groups: G_1 (pixels $> T$) and G_2 (pixels $\leq T$).
3. **Mean Computation:** Calculate the average intensity values m_1 and m_2 for groups G_1 and G_2 respectively.
4. **Update:** Compute a new threshold $T = \frac{1}{2}(m_1 + m_2)$.
5. **Iteration:** Repeat steps 2 through 4 until T stabilizes (the change is smaller than a predefined epsilon).

5.3.2 Numerical Example

Suppose we have a set of 16 pixel intensities:

$$\{10, 12, 15, 20, 22, 25, 100, 110, 120, 130, 140, 150, 160, 170, 180, 200\}$$

Iteration 1:

- **Initial T_0:** Global mean ≈ 101.
- **Groups:** $G_1 = \{110, \dots, 200\}$ (9 pixels), $G_2 = \{10, \dots, 100\}$ (7 pixels).
- **Means:** $m_1 = 151.1$, $m_2 = 29.1$.
- **New T:** $\frac{1}{2}(151.1 + 29.1) = 90.1$.

Iteration 2:

- **Thresholding with $T = 90.1$:** The value 100 now moves from G_2 to G_1.
- **New Groups:** $G_1 = \{100, 110, \dots, 200\}$ (10 pixels), $G_2 = \{10, \dots, 25\}$ (6 pixels).
- **New Means:** $m_1 = 146$, $m_2 = 17.3$.
- **New T:** $\frac{1}{2}(146 + 17.3) = 81.65$.

The algorithm continues until the threshold value converges, effectively separating the dark cluster from the light cluster.

6 Otsu's Method for Global Thresholding

While iterative methods rely on heuristic averaging, Otsu's method is grounded in statistical discriminant analysis. Its objective is to segment foreground and background by maximizing the **between-class variance**. Computations are performed on the normalized image histogram.

6.1 Mathematical Formulation

Let the image have intensity levels from 0 to $L-1$. The normalized histogram gives the probability p_i of each intensity level i.

We define a threshold k $(0 < k < L-1)$ that divides pixels into two classes:

- C_1: Pixels with intensities $[0, k]$
- C_2: Pixels with intensities $[k+1, L-1]$

The probabilities of a pixel belonging to C_1 or C_2 are:

$$P_1(k) = \sum_{i=0}^{k} p_i \tag{3}$$

$$P_2(k) = \sum_{i=k+1}^{L-1} p_i = 1 - P_1(k) \tag{4}$$

The mean intensities of the classes are:

$$m_1(k) = \frac{1}{P_1(k)} \sum_{i=0}^{k} i p_i \tag{5}$$

$$m_2(k) = \frac{1}{P_2(k)} \sum_{i=k+1}^{L-1} i p_i \tag{6}$$

The global mean of the entire image is $m_G = \sum_{i=0}^{L-1} i p_i$.

6.2 Optimization Criterion

Otsu's method finds the optimal threshold k^* that maximizes the between-class variance $\sigma_B^2(k)$:

$$k^* = \arg \max_{0 \le k \le L-1} \sigma_B^2(k) \tag{7}$$

Where the between-class variance is defined as:

$$\sigma_B^2(k) = P_1(k)(m_1(k) - m_G)^2 + P_2(k)(m_2(k) - m_G)^2 \tag{8}$$

6.3 Numerical Example

Consider a 10-pixel image with 4 intensity levels ($L = 4$). Pixel values: $\{0, 0, 0, 1, 1, 2, 3, 3, 3, 3\}$.

Probabilities: $p_0 = 0.3$, $p_1 = 0.2$, $p_2 = 0.1$, $p_3 = 0.4$. Global Mean: $m_G = 0(0.3) + 1(0.2) + 2(0.1) + 3(0.4) = 1.6$.

We evaluate the variance for potential thresholds k:

- **For $k = 0$:** $P_1 = 0.3, P_2 = 0.7, m_1 = 0, m_2 \approx 2.28$.
  $\sigma_B^2(0) = 0.3(0 - 1.6)^2 + 0.7(2.28 - 1.6)^2 \approx 1.09$.
- **For $k = 1$:** $P_1 = 0.5, P_2 = 0.5, m_1 = 0.4, m_2 = 2.8$.
  $\sigma_B^2(1) = 0.5(0.4 - 1.6)^2 + 0.5(2.8 - 1.6)^2 = 1.44$.
- **For $k = 2$:** $P_1 = 0.6, P_2 = 0.4, m_1 \approx 0.67, m_2 = 3$.
  $\sigma_B^2(2) = 0.6(0.67 - 1.6)^2 + 0.4(3 - 1.6)^2 \approx 1.30$.

The maximum variance is 1.44 at $k = 1$. Therefore, the optimal threshold is $T = 1$.

7 Improving Thresholding

Global thresholding can fail if the image is noisy or if lighting is uneven.

7.1 Image Smoothing

Noisy images produce jagged histograms where it is difficult to find a clear valley. Applying Gaussian smoothing prior to thresholding smooths the histogram, making global thresholding algorithms (like Otsu's) significantly more effective.

7.2 Variable (Local) Thresholding

When an image has uneven illumination (e.g., shadows across a document), a single global threshold will fail. Variable thresholding solves this by subdividing the image or computing a unique threshold T_{xy} for every pixel based on its local neighborhood.

A common approach computes the threshold based on local mean (m_{xy}) and standard deviation (σ_{xy}):

$$T_{xy} = a\sigma_{xy} + bm_{xy} \quad (a, b \geq 0) \tag{9}$$

For text segmentation, a **moving average** approach is highly effective. It calculates the threshold $T_{xy} = bm_{xy}$ where m is estimated using only the last n visited pixels along a scan line.

8 Color Segmentation and K-Means Clustering

When moving from grayscale to color images, the feature space expands to multiple dimensions (e.g., 3D spaces like RGB or HSV). For generic color segmentation where no prior target color is specified, clustering methods like K-means are highly effective.

8.1 K-Means Clustering: The Concept

K-means is an unsupervised clustering algorithm. In the context of image segmentation, it plots every pixel as a point in a multidimensional color space and attempts to group these points into K distinct clusters based on their proximity (color similarity).

8.2 The Mathematical Objective

Given a set of image pixels $(x_1, \ldots, x_n)$, the goal is to partition the data into K sets, $S = (S_1, \ldots, S_K)$, to minimize the within-cluster variance. The objective function is:

$$\arg\min_{S} \sum_{i=1}^{K} \sum_{x \in S_i} ||x - \mu_i||^2 \tag{10}$$

Where:

- K is the predefined number of clusters.
- x is the feature vector of a pixel (e.g., its RGB values).
- μ_i is the centroid (mean vector) of cluster S_i.

8.3 The Iterative Algorithm

K-means minimizes the objective function through the following iterative steps:

1. **Random Initialization:** Select K random points in the feature space to serve as the initial centroids.
2. **Points Association:** Calculate the distance between every pixel x and all K centroids. Assign each pixel to the cluster with the nearest centroid.

3. **Centroids Update:** Recalculate the position of the K centroids by taking the mean of all pixels currently assigned to each cluster.
4. **Convergence Check:** Repeat Steps 2 and 3 until stability is reached (i.e., pixel assignments no longer change or centroid movement falls below a threshold).

8.4 Numerical Example: RGB Clustering

Consider an image with 6 pixels and $K = 2$ clusters.

1. Data Points (RGB):

- $P_1(200, 0, 0), P_2(180, 20, 20), P_3(220, 10, 10)$ (Reddish)
- $P_4(0, 150, 0), P_5(10, 170, 10), P_6(20, 130, 20)$ (Greenish)

2. Initialization: Assume initial centroids $C_1 = P_1(200, 0, 0)$ and $C_2 = P_4(0, 150, 0)$.

3. Assignment Step (Iteration 1): Using Euclidean distance $D = \sqrt{\Delta R^2 + \Delta G^2 + \Delta B^2}$:

- For P_2: $Dist(P_2, C_1) \approx 34.6$ vs $Dist(P_2, C_2) \approx 222.9 \rightarrow$ Cluster 1.
- For P_5: $Dist(P_5, C_1) \approx 255.1$ vs $Dist(P_5, C_2) \approx 24.5 \rightarrow$ Cluster 2.

After checking all points, Cluster 1 contains $\{P_1, P_2, P_3\}$ and Cluster 2 contains $\{P_4, P_5, P_6\}$.

4. Update Step: Calculate new centroids by taking the mean of assigned pixels:

- $C_{1\_new} = \left(\frac{200+180+220}{3}, \frac{0+20+10}{3}, \frac{0+20+10}{3}\right) = (200, 10, 10)$
- $C_{2\_new} = \left(\frac{0+10+20}{3}, \frac{150+170+130}{3}, \frac{0+10+20}{3}\right) = (10, 150, 10)$

The centroids have moved closer to the center of their respective color clouds. The process repeats until assignments no longer change.

8.5 Extended Feature Space: RGBXY

Standard K-means on RGB values groups pixels strictly by color, ignoring their physical location in the image. A red pixel in the top-left corner and a red pixel in the bottom-right corner will be placed in the same cluster. To encourage spatially contiguous segments, the feature space can be expanded to 5 dimensions: **RGBXY**. This forces the algorithm to consider both color similarity and spatial proximity when calculating the distance.

8.6 Pros, Cons, and the Choice of K

- **Pros:** The algorithm is simple to implement, understand, and computationally relatively fast.
- **Cons:** Results can be non-deterministic due to the random initialization of centroids. Furthermore, it does not inherently prevent the formation of highly unbalanced clusters.
- **Choosing K:** The number of clusters K must be set a priori. A common solution to finding the optimal K is to run the algorithm across multiple values of K (e.g., $K = 2, 3, 4, 5$) and evaluate the results using a cluster quality metric, such as the **Davies-Bouldin index**, to find the partition that yields the most distinct and cohesive clusters.

9 Superpixels and the SLIC Algorithm

Superpixels provide a convenient primitive from which to compute local image features. They group pixels into compact, roughly uniform regions that respect object boundaries, capturing redundancy in the image and drastically reducing the complexity of subsequent computer vision tasks.

9.1 SLIC: Simple Linear Iterative Clustering

SLIC is an adaptation of the K-means algorithm designed specifically for generating superpixels. It performs a local clustering of pixels in a 5-dimensional space defined by $[l, a, b, x, y]$.

- l, a, b: The pixel color vector in the CIELAB color space, chosen for its perceptual uniformity.
- x, y: The spatial position of the pixel in the image.

9.2 The Distance Metric (D_s)

Because CIELAB color distances and spatial plane distances have different scales, they cannot be directly combined using a simple Euclidean norm. SLIC introduces a specialized distance measure D_s that normalizes the spatial distance.

Given a cluster center $C_k = [l_k, a_k, b_k, x_k, y_k]^T$ and a pixel i at $[l_i, a_i, b_i, x_i, y_i]^T$, the distances are computed as follows:

$$d_{lab} = \sqrt{(l_k - l_i)^2 + (a_k - a_i)^2 + (b_k - b_i)^2} \tag{11}$$

$$d_{xy} = \sqrt{(x_k - x_i)^2 + (y_k - y_i)^2} \tag{12}$$

$$D_s = d_{lab} + \frac{m}{S} d_{xy} \tag{13}$$

Parameters:

- K: The desired number of superpixels (the only required user input).
- S: The regular grid interval between superpixels, defined as $S = \sqrt{N/K}$ where N is the total number of pixels.
- m: The compactness parameter. A higher m emphasizes spatial proximity, resulting in more compact, rigidly square superpixels.

9.3 Algorithm Complexity ($O(N)$)

Standard K-means evaluates distances from every pixel to every cluster center, resulting in $O(NKI)$ complexity. SLIC drastically reduces this by localizing the search. Since a superpixel occupies roughly S^2 area, SLIC restricts the pixel search to a $2S \times 2S$ region surrounding the cluster center. Consequently, a pixel is evaluated against no more than eight centers, granting the algorithm a strictly linear $O(N)$ complexity, regardless of K.

9.4 Algorithm Steps

1. **Initialization:** Sample K cluster centers at regular grid intervals S. Initialize distance $d(i) = \infty$ and label $l(i) = -1$ for all pixels.
2. **Perturbation:** Move centers to the lowest gradient position in a 3×3 neighborhood to avoid noisy edges.
3. **Assignment:** For each center k, evaluate pixels in a $2S \times 2S$ region. Compute D_s. If $D_s < d(i)$, update $d(i) = D_s$ and assign the label $l(i) = k$.
4. **Update:** Recompute centers as the mean $[l, a, b, x, y]$ vector of all associated pixels.
5. **Convergence:** Repeat assignment and update until residual error converges.
6. **Connectivity:** Enforce spatial connectivity by merging small, stray, or disjoint segments with the largest neighboring cluster.

9.5 Numerical Example: Label Assignment

Suppose an image has $N = 100$ pixels and we want $K = 4$ superpixels.

- Grid interval $S = \sqrt{100/4} = 5$.
- Compactness parameter $m = 10$.

Target Pixel P_i: Located at $(4, 5)$ with color $(60, 0, 0)$. Initial state: $l(i) = -1$, $d(i) = \infty$.
Testing Cluster Center 1 (C_1): Located at $(2, 2)$ with color $(50, 0, 0)$.

$$d_{lab} = \sqrt{(50 - 60)^2} = 10$$

$$d_{xy} = \sqrt{(2 - 4)^2 + (2 - 5)^2} = \sqrt{13} \approx 3.61$$

$$D_s = 10 + \left(\frac{10}{5}\right) \times 3.61 = 17.22$$

Since $17.22 < \infty$, update pixel: $d(i) = 17.22$, $l(i) = 1$.

Testing Cluster Center 2 (C_2): Located at $(7, 5)$ with color $(65, 0, 0)$.

$$d_{lab} = \sqrt{(65-60)^2} = 5$$
$$d_{xy} = \sqrt{(7-4)^2 + (5-5)^2} = \sqrt{9} = 3$$
$$D_s = 5 + \left(\frac{10}{5}\right) \times 3 = 11$$

Since $11 < 17.22$, update pixel: $d(i) = 11$, $l(i) = 2$.

Pixel P_i successfully finds its nearest center and is assigned to Cluster 2.

10 The Goal of Image Matching

The core problem introduced in this section is determining how similar two images are, which is highly dependent on the application. The course breaks this down into two main approaches:

- **Global Matching:** This involves computing a single, overarching descriptor for the entire image, such as a color histogram or overall brightness mean. This is typically used for general tasks like image retrieval or broad image classification.
- **Local Matching:** Instead of looking at the whole image, this method focuses on finding specific, localized "points of interest" and matching them across different images. This is crucial for precise tasks like 3D reconstruction and image registration.

10.1 The Local Matching Pipeline

We focuses heavily on the Local Matching approach. The standard three-step pipeline forms the backbone of the rest of the course:

1. **Feature Detection:** Finding the specific "interest points" (like corners or blobs) on each image.
2. **Feature Description:** Computing a mathematical vector that describes the visual neighborhood around each detected point.
3. **Feature Matching:** Comparing these descriptions between two images to find the matching pairs.

10.2 Degrees of Difficulty in Matching

Local matching ranges from "easy" to "much harder" based on the transformations between the two images:

- **Easy:** The images only have simple translations (shifting up/down/left/right).
- **Harder:** The images undergo translation, changes in illumination, and slight deformations.
- **Much Harder:** The images feature dramatic changes in the 3D viewpoint and heavy illumination changes.

10.3 Integration from the DoG and SIFT Paper

To solve these "much harder" cases, like the NASA Mars Rover images, we cannot rely on simple pixel comparisons. David Lowe's paper introduces **SIFT (Scale-Invariant Feature Transform)** specifically to address this. The goal is to transform image data into coordinates that are invariant to image scaling and rotation, and highly robust against 3D viewpoint changes and lighting variations. This allows a single local feature to find its correct match even in a massive database of features.

11 Scale Invariant Feature detectors

In Chapter 1, we established the need to find "interest points" (like corners or blobs). However, simple corner detectors fail when the distance to the object changes. A leaf might look like a tiny blob from 100 meters away, but show sharp corners from 1 meter away. To perform robust matching, we need an algorithm that finds features regardless of how zoomed in or out the image is. David Lowe's SIFT (Scale-Invariant Feature Transform) solves this.

11.1 Building "Scale-Space"

To teach a computer to look at an image at different scales, we blur it. Blurring removes high-frequency details, simulating how an object appears from further away.

The "scale space" of an image is a function $L(x, y, \sigma)$, produced by convolving the original image $I(x, y)$ with a 2D Gaussian blur filter $G(x, y, \sigma)$:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y) \tag{14}$$

Where σ (sigma) is the scale (amount of blur).

11.2 The Blob Finder: Laplacian of Gaussian (LoG)

To find interesting features at these scales, we look for "blobs" (uniform areas surrounded by sharp variations). The mathematical tool for finding blobs is the Laplacian, which measures the 2nd spatial derivative of the image.

$$\nabla^2 L = \frac{\partial^2 L}{\partial x^2} + \frac{\partial^2 L}{\partial y^2} \tag{15}$$

11.2.1 Numerical Example: 1D LoG

Imagine a 1D row of pixel intensities with a bright blob in the center:

$$I = [0, 0, 10, 10, 0, 0]$$

The 1st derivative (difference between adjacent pixels) highlights the edges:

$$I' = [0, 10, 0, -10, 0]$$

The 2nd derivative (difference of differences) highlights the center of the peak and the surrounding dip:

$$I'' = [10, -10, -10, 10]$$

When applied in 2D to a blurred image, this filter looks like a "Mexican hat". The algorithm searches for the specific σ where this 2nd derivative response is absolutely maximized.

11.3 The Genius Shortcut: Difference of Gaussians (DoG)

Calculating the 2nd derivative for every pixel at every possible scale is computationally expensive. Lowe's SIFT provides an optimization based on the heat diffusion equation: subtracting two images with different levels of Gaussian blur closely approximates the normalized Laplacian of Gaussian.

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \tag{16}$$

11.4 How Different Should the Two Scales Be? (The multiplier k)

This is a crucial parameter in SIFT. We don't just pick random blur levels. The scale space is divided into "octaves". An octave corresponds to doubling the value of σ. Within each octave, Lowe divides the space into an integer number of intervals, s. The scale multiplier k between two adjacent blurred images is defined as:

$$k = 2^{1/s} \tag{17}$$

In his foundational paper, Lowe recommends $s = 3$ intervals per octave. Therefore, the difference in scale between two adjacent blurred images is $k = 2^{1/3} \approx 1.2599$. If your base scale is $\sigma = 1.6$, the next image in the stack is blurred at $\sigma = 1.6 \times 1.26 = 2.016$.

11.4.1 Numerical Example: DoG Calculation

Suppose at pixel coordinate $(50, 50)$, the intensity in the first blurred image $L(x, y, \sigma)$ is 145. In the next progressively blurred image $L(x, y, k\sigma)$, the intensity is 130. The DoG value for that pixel at that scale is:

$$D(50, 50, \sigma) = 130 - 145 = -15 \tag{18}$$

11.5 Extrema Detection (Finding the Keypoints)

Once we have a 3D stack of DoG images (X, Y, and Scale), we look for local extrema (maxima or minima) to identify keypoints.

A single pixel is compared against its 26 neighbors in the 3D scale-space:

- 8 adjacent pixels in its current scale ($k\sigma$).
- 9 pixels directly above it in the next scale ($k^2\sigma$).
- 9 pixels directly below it in the previous scale (σ).

11.6 Numerical Example: 3D Neighborhood Check

Imagine a $3 \times 3 \times 3$ cube of DoG values. The center pixel is our candidate.

- **Scale σ (Below):** All 9 pixels have values ranging from -5 to 12.
- **Scale $k\sigma$ (Current):** The center candidate has a value of **45**. Its 8 immediate neighbors range from 20 to 38.
- **Scale $k^2\sigma$ (Above):** All 9 pixels have values ranging from 15 to 30.

Because 45 is strictly greater than all 26 surrounding values, this coordinate $(x, y, k\sigma)$ is officially recorded as a keypoint!

11.7 Filtering Bad Keypoints (Additional Info)

Simply finding extrema isn't enough; some points are unstable. SIFT applies two more filters:

1. **Low Contrast Filter:** If the absolute DoG value at the extremum is less than a threshold (Lowe uses 0.03), it is discarded as noise.
2. **Edge Response Elimination:** DoG has a strong response along flat edges, which are terrible for matching (the aperture problem). SIFT computes the Hessian matrix (2nd derivatives in x and y) at the keypoint. If the ratio of the principal curvatures indicates it's an edge rather than a blob, the point is discarded.

12 Feature descriptors

In Chapter 2, we found the (x, y) coordinates and scale (σ) of our keypoints. To match them, we need to describe what the visual area around them looks like.

12.1 The Problem with "Raw Patches"

The simplest method is to take a square window (a patch) of pixels around the keypoint and list the pixel intensities. However, if the camera rotates even slightly between two images, the pixels in that square window will completely shift. A pixel-by-pixel comparison (Sum of Squared Differences) will fail. We need a descriptor that is **invariant to rotation**.

12.2 Clarifying the Keypoint Neighborhood

A common confusion is assuming the descriptor is built from a single pixel. A keypoint is just a central coordinate. SIFT builds its descriptor by analyzing a **neighborhood** of pixels around that center point. The size of this neighborhood depends on the keypoint's characteristic scale (σ) found in Chapter 2.

For every pixel in this neighborhood, the computer calculates two values using finite differences:

- **Gradient Magnitude** (m)**:** How sharp the contrast is (edge strength).
- **Gradient Orientation** (θ)**:** Which direction is the brightest (from 0° to 360°).

12.3 Finding "North": The 36-Bin Orientation Histogram

To make our descriptor immune to rotation, SIFT gives each keypoint its own distinct "North" based on the visual data around it.

12.3.1 How the 36 Bins Work

The algorithm creates a histogram with 36 bins. Since a full circle is 360°, each bin covers exactly 10°.

- Bin 1: 0° to 9°
- Bin 2: 10° to 19°
- ...
- Bin 36: 350° to 359°

12.3.2 Numerical Example: Voting for an Orientation

Imagine the neighborhood around our keypoint has 50 pixels. We calculate the angle and magnitude for each:

- **Pixel A:** $\theta = 12^\circ$, Magnitude $= 5$. It belongs to **Bin 2**. We add 5 to Bin 2.
- **Pixel B:** $\theta = 18^\circ$, Magnitude $= 10$. It also belongs to **Bin 2**. We add 10. (Bin 2 is now 15).
- **Pixel C:** $\theta = 95^\circ$, Magnitude $= 2$. It belongs to **Bin 10**. We add 2 to Bin 10.

After processing all 50 pixels, we look at the 36 bins. Suppose **Bin 5** $(40^\circ - 49^\circ)$ has the highest total magnitude (a massive spike in the histogram). This means the dominant visual feature around this keypoint is pointing at roughly 45°. SIFT now rotates the entire coordinate system of this patch by 45°. **The patch is now rotationally invariant!**

12.4 Building the 128-Dimensional SIFT Descriptor

Now that the patch is scaled and rotated correctly, we build the actual mathematical fingerprint.

12.4.1 The Grid and the 8 Bins

1. Take a 16×16 **window** of pixels centered on the keypoint.
2. Divide this window into a 4×4 **grid**. This gives us 16 distinct "sub-regions", each containing $4 \times 4 = 16$ pixels.
3. Inside each of the 16 sub-regions, we create a new, smaller histogram. This one only has **8 bins** (each covering 45°: N, NE, E, SE, S, SW, W, NW).
4. We tally up the gradients of the 16 pixels into these 8 bins.

12.4.2 What Information is in the 8 Bins?

The 8 bins describe the geometry of the lines inside that specific sub-region. For example, look at the top-left sub-region. If it contains a sharp horizontal edge, the gradients will point straight up or straight down. Therefore, the "North" (90°) and "South" (270°) bins will have huge numbers, while the other 6 bins will be close to zero.

12.4.3 The Final Vector

You have 16 sub-regions. Every sub-region gives you 8 numbers (the 8 bins).

16 sub-regions × 8 bins = 128 numbers.

The final 128-dimensional descriptor is literally just a list of these numbers: $[v_1, v_2, v_3, ..., v_{128}]$. It tells the computer exactly how edges are spatially arranged around the keypoint.

12.5 Solving Illumination Invariance

What if one image was taken in bright daylight and the other at dusk? The 128 numbers will be drastically different because the gradient magnitudes will be smaller in the dark image. To fix this, we normalize the 128-dimensional vector to have a "unit length" of 1:

$$\hat{v} = \frac{v}{\sqrt{v_1^2 + v_2^2 + ... + v_{128}^2}} \tag{19}$$

By doing this, global contrast changes are cancelled out. SIFT also caps any single value at 0.2 and re-normalizes to protect against harsh, non-linear camera saturation.

13 Introduction to Feature Matching

Once local features are detected (Chapter 2) and described (Chapter 3), the final step in the pipeline is feature matching: finding the one-to-one correspondences between features in Image 1 and Image 2. This requires defining a **Similarity Measure** (how to score the match mathematically) and a **Matching Strategy** (how to decide which matches to keep).

13.1 Similarity Measures for Image Patches

When using raw image patches of size $W \times W$ (denoted as $N1$ and $N2$) instead of SIFT descriptors, two primary mathematical methods are used to compute their similarity.

13.1.1 Sum of Squared Differences (SSD)

SSD calculates the direct pixel-by-pixel intensity difference:

$$\phi_{SSD}(N1, N2) = -\sum_{k,l=-\frac{W}{2}}^{\frac{W}{2}} (N1(k,l) - N2(k,l))^2 \tag{20}$$

The negative sign is applied so that a perfect match results in the maximum possible score (0), while increasingly dissimilar patches yield highly negative scores.

Numerical Example: For two 2×2 patches:

$$N1 = \begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix}, N2 = \begin{bmatrix} 12 & 18 \\ 10 & 20 \end{bmatrix} \; SSD = -[(10-12)^2+(20-18)^2+(10-10)^2+(20-20)^2] = -[4+4+0+0] = \mathbf{-8}$$

13.1.2 Normalized Cross Correlation (NCC)

SSD is highly sensitive to lighting changes. NCC corrects for affine changes in illumination by normalizing the pixels against the patch's mean (μ) and standard deviation (σ):

$$\phi_{NCC}(N1, N2) = \frac{\sum_{k,l=-\frac{W}{2}}^{\frac{W}{2}} (N1(k,l) - \mu_1)(N2(k,l) - \mu_2)}{W^2 \sigma_1 \sigma_2} \tag{21}$$

Where μ_i is the mean brightness of patch i, and σ_i is its standard deviation. This function returns a score exactly in the range $[-1, 1]$, where 1 is a perfect match. By subtracting the mean, a bright image and a dark image of the exact same object will perfectly match.

13.2 Similarity Measures for Histograms

When matching SIFT descriptors (which are 128-bin histograms), Euclidean distance is standard, but the course also highlights **Histogram Intersection**:

$$HI(H^1, H^2) = \sum_{i=1}^{N_{bin}} \min(H_i^1, H_i^2) \tag{22}$$

This measures the overlapping area between two histograms.

Numerical Example: Let $H^1 = [10, \mathbf{5}, \mathbf{2}]$ and $H^2 = [\mathbf{8}, 20, 3]$. The intersection is $\min(10, 8) + \min(5, 20) + \min(2, 3) = 8 + 5 + 2 = \mathbf{15}$.

13.3 Matching Strategy: Lowe's Ratio Test

A Brute Force approach checks every feature against every other feature and picks the minimum distance. However, in environments with repeating patterns (like windows on a building), multiple features will look mathematically identical, leading to false matches.

David Lowe introduced the **Ratio Test**:

$$Ratio = \frac{\text{Distance to Best Match}}{\text{Distance to Second Best Match}} \tag{23}$$

If the ratio is low (e.g., 0.1), the best match is uniquely excellent. If the ratio is high (e.g., 0.9), it means the best match and second-best match are almost identical, indicating an ambiguous match. SIFT strictly rejects any match where this ratio is > 0.8.

13.4 Matching Through an Affinity Matrix

An Affinity Matrix A represents the similarity between all pairs of features, where $A(i, j)$ is the score between feature i in Image 1 and feature j in Image 2. This approach can combine spatial location and visual appearance.

13.4.1 Spatial Vicinity (Distance Matrix E)

$$E(i, j) = e^{-\frac{||p_i^1 - p_j^2||^2}{2\sigma^2}} \tag{24}$$

Here, p_i^1 and p_j^2 are the spatial (X, Y) coordinates of the keypoints. σ is a tolerance parameter dictating how much physical movement is allowed. If a point moves minimally, the squared distance $||p_i^1 - p_j^2||^2$ is small, and $e^0 \approx 1$.

13.4.2 Combined Affinity Matrix (A)

To combine the spatial score E with the appearance score NCC, the formula is:

$$A(i, j) = E(i, j) \times \frac{1}{2}(\phi_{NCC}(Q_i^1, Q_j^2) + 1) \tag{25}$$

The expression $\frac{1}{2}(NCC + 1)$ shifts the NCC score from $[-1, 1]$ to $[0, 1]$. Therefore, an entry in A will only approach 1.0 if the features are physically close AND look visually identical.

13.4.3 Extracting Matches from A

To guarantee a 1-to-1 correspondence, a match at (i, j) is only accepted if:

1. $A(i, j)$ is the maximum value in row i.
2. $A(i, j)$ is the maximum value in column j.
3. $A(i, j) >$ threshold.

14 Introduction to Motion Analysis

Motion analysis involves analyzing video sequences rather than static images to focus on motion information. This adds a temporal dimension, allowing the visual system to perceive structures and details that might be invisible in a single frame.

14.1 The Image Sequence

An image sequence is defined as a series of N images, or frames, acquired at discrete time instants. The mathematical representation of frame timing is:

$$t_k = t_0 + k\Delta t \tag{26}$$

Where:

- t_k: Time of the current frame k.
- t_0: The starting time.
- Δt: The fixed (usually small) time interval between acquisitions (e.g., 1/30s for standard video).

14.2 Brightness Constancy Assumption

A fundamental assumption in motion analysis is that the illumination of the scene does not vary significantly within the observation interval. Under this assumption, changes in pixel intensity from time t_1 to t_2 are attributed solely to the relative motion between the scene and the camera.

14.3 Core Problems in Motion Analysis

Motion analysis can be categorized into several distinct computational problems:

- **Correspondence**: Identifying which elements of one frame correspond to which elements of the next frame. This is closely related to image matching.
- **Reconstruction**: Determining the 3D position and 3D velocity of observed elements from their 2D motion on the image plane.
- **Motion Segmentation**: Partitioning the image into regions corresponding to different moving parts.
- **Tracking**: Estimating the trajectories (2D or 3D) of specific points or objects over a sequence of frames using tools like dynamic filters (e.g., Kalman filters).

14.4 Methodological Summary

The course focuses on the following low-level algorithms and temporal features:

1. **Change Detection**: Used for motion segmentation when the camera is stationary.
2. **Optical Flow**: Addressing the correspondence problem to estimate pixel-wise motion.
3. **Feature Tracking**: Following specific image points (like corners) through the sequence.

15 Motion Segmentation: Change Detection

When the camera is stationary, motion segmentation is implemented as a difference between the current frame and a reference model.

15.1 Basic Thresholding

The threshold τ is used to separate motion from noise.

$$M_t(x,y) = \begin{cases} 1 & \text{if } |I_t(x,y) - I_{ref}(x,y)| > \tau \\ 0 & \text{otherwise} \end{cases} \tag{27}$$

The threshold τ depends on acquisition conditions and sensor noise.

15.1.1 Statistical Basis

In an empty scene, the difference $|I_t - I_{t+1}|$ follows a Quasi-Gaussian distribution caused by sensor noise and illumination instability (e.g., neon flickering).

- **Stable Conditions**: Narrow distribution. Typically $\tau \approx 10$.
- **Unstable Conditions**: Wide distribution. Typically $\tau \approx 30$.

15.2 Background Modeling Versions

15.2.1 Version 1: Static Average

The reference image is the average of the first N frames:

$$B = \frac{1}{N} \sum_{t=1}^{N} I_t \tag{28}$$

Numerical Example: For $N = 2$, if a pixel (x, y) has intensities $I_1 = 100$ and $I_2 = 110$, then $B = 105$. This value is fixed.

15.2.2 Version 2: Running Average (Temporal Filtering)

Handles slow illumination changes by updating the background model B_t at each frame using a learning rate α:

$$B_t = (1 - \alpha)B_{t-1} + \alpha I_t \tag{29}$$

Numerical Example: If $B_{t-1} = 100$, $I_t = 120$, and $\alpha = 0.05$: $B_t = (0.95 \times 100) + (0.05 \times 120) = 95 + 6 = 101$.

15.2.3 Version 3: Selective Update for Abrupt Variations

Designed to handle persistent changes (e.g., a moved object). It compares the current frame to a frame from Δt steps ago to check for persistence.

$$B_t = \begin{cases} B_{t-1} & \text{if } |I_t - I_{t-\Delta t}| > \tau' \\ (1 - \alpha)B_{t-1} + \alpha I_t & \text{otherwise} \end{cases} \tag{30}$$

Numerical Example: A person walks through a pixel. $|I_t - I_{t-\Delta t}|$ is high (e.g., 50). The background is NOT updated ($B_t = B_{t-1}$), preventing the "ghosting" of the person into the background.

15.3 Blob Analysis and Descriptors

After thresholding, pixels are grouped into **Blobs** via Connected Components. A blob is a segment where any two pixels can be connected by a path remaining inside the segment.

15.3.1 Common Descriptors

- **Centroid $(\bar{x}, \bar{y})$**: $\bar{x} = \frac{1}{A}\sum x_i, \bar{y} = \frac{1}{A}\sum y_i$, where A is Area.
- **Area**: Total number of pixels in the blob.
- **Velocity**: $\vec{v} = (v_x, v_y)$, representing the displacement magnitude and direction.
- **Higher Order Moments**: Used for shape description and orientation.

15.4 Recap

1. **Static Average**: $B = \frac{1}{N} \sum I_t$. (Assumes constant background).
2. **Running Average**: $B_t = (1 - \alpha)B_{t-1} + \alpha I_t$. (Handles slow illumination changes).
3. **Selective Persistent Update**: Only updates the background if the change is persistent ($|I_t - I_{t-\Delta t}|$ is small), preventing moving objects from being incorporated into the background.

16 Correspondence: Optical Flow

Correspondence identifies matching elements across frames acquired close in time ($t_k = t_0 + k\Delta t$). This allows us to calculate **Optical Flow**: the apparent motion of brightness patterns.

16.1 The Concept of Optical Flow

In the context of motion, correspondence can be viewed as estimating the **apparent motion** of the brightness pattern, known as **Optical Flow**.

Why apprant? Optical flow is the motion we perceive, which may not always match the true 3D physical motion of the object. The slides provide two classic examples:

- **The Bouncing Ball**: A uniform, textureless ball bouncing in a dark room. Because the pixels don't change color or intensity as the ball moves, the visual system perceives nothing—the optical flow is zero despite the physical motion.
- **The Barber Pole**: As a barber pole rotates (3D rotational motion), we perceive the stripes moving vertically (apparent motion). This illustrates the difference between the motion field (the 2D projection of 3D motion) and the optical flow.

16.2 Correlation-Based Approaches

This is a "search-based" method for finding correspondence. Instead of using calculus, we look for matching pixel patterns. **The Algorithm:**

1. **Select a Feature**: Identify a meaningful point (like a corner) in image I_t and define a surrounding "patch" or neighborhood N_1.
2. **Define a Search Region**: Define a small region R in the next frame I_{t+1} where you expect the point to have moved.
3. **Find the Best Match**: Slide the patch N_1 over the search region R and compare it to candidate patches N_2 using a similarity measure.
4. **Estimate Motion**: The distance between the original position of N_1 and the position of the most similar patch N_2^* is the local estimate of apparent motion.

16.2.1 Similarity Measures

We measure how "close" two patches are pixel-by-pixel using mathematical functions. Two common measures are:

1. Sum of Squared Differences (SSD):

$$\phi_{SSD}(N1, N2) = -\sum_{k,l}(N1(k,l) - N2(k,l))^2 \tag{31}$$

- **Logic**: We subtract the intensity of each pixel in N_2 from N_1, square the result (to make all differences positive), and sum them.
- **Goal**: The best match is the one where the sum is closest to zero (represented here with a negative sign so that a "higher" value near 0 is better).

2. Normalized Cross Correlation (NCC):

$$\phi_{NCC}(N1, N2) = \sum_{k,l} \frac{(N1(k,l) - \mu_1)(N2(k,l) - \mu_2)}{W^2 \sigma_1 \sigma_2} \tag{32}$$

- μ **(Mean)**: The average brightness of the patch.
- σ **(Standard Deviation)**: The contrast or variation in the patch.
- **Logic**: By subtracting the mean, this measure is robust to global lighting changes (e.g., the whole image getting slightly brighter).
- **Range**: The result is always between $[-1, 1]$, where 1 is a perfect match.

16.2.2 Numerical Example: SSD Calculation

Given patch $N_1 = [10, 20; 10, 20]$ and candidate $N_2 = [12, 22; 12, 22]$:

1. Differences: $(10 - 12) = -2, (20 - 22) = -2, \ldots$
2. Squares: $(-2)^2 = 4$
3. Sum: $4 + 4 + 4 + 4 = 16$
4. $\phi_{SSD} = -16$

16.3 Gradient-Based Optical Flow

The Brightness Constancy Equation assumes $I(x, y, t) = I(x + u, y + v, t + 1)$. By Taylor expansion, we obtain the **Optical Flow Constraint Equation**:

$$I_x u + I_y v + I_t = 0 \tag{33}$$

where I_x, I_y are spatial gradients and I_t is the temporal gradient.

16.3.1 The Aperture Problem

One equation with two unknowns (u, v) is underdetermined. We can only compute motion in the direction of the gradient (**Normal Flow**):

$$u_n = \frac{-I_t}{\sqrt{I_x^2 + I_y^2}} \tag{34}$$

16.4 The Lucas-Kanade Algorithm

By assuming velocity $\mathbf{u}$ is constant over a neighborhood N of m pixels, we solve the overdetermined system $A\mathbf{u} = \mathbf{b}$ using the pseudo-inverse:

$$\mathbf{u} = (A^\top A)^{-1} A^\top \mathbf{b} \tag{35}$$

where:

$$A = \begin{bmatrix} I_x(p_1) & I_y(p_1) \\ \vdots & \vdots \\ I_x(p_m) & I_y(p_m) \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} -I_t(p_1) \\ \vdots \\ -I_t(p_m) \end{bmatrix} \tag{36}$$

16.5 Multi-Resolution Pyramids

To handle large displacements where Taylor approximations fail, we use **Gaussian Pyramids**.

1. Estimate flow at a coarse (downsampled) level.
2. Upsample the flow to the next level: $u(i) = 2 \times u(i - 1)$.
3. **Warp** frame $I(t)$ using the current estimate.
4. Calculate the local correction $u'(i)$ using LK on the warped image.
5. Final Flow: $u_{final} = \sum u_{corrections}$.

16.5.1 Numerical Examples

1. LK System: If $A^\top A = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}$ and $A^\top \mathbf{b} = \begin{bmatrix} 20 \\ 50 \end{bmatrix}$, then: $\mathbf{u} = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix} \begin{bmatrix} 20 \\ 50 \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$.

2. Pyramid Upsampling: If a pixel moves 3 pixels at level $i = 1$ (half resolution), it corresponds to a 6-pixel move at level $i = 0$ (full resolution).

17 Correspondence Over Time: Feature Tracking

While optical flow calculates motion between two consecutive frames, **Feature Tracking** stitches these frame-to-frame movements together to form long-term trajectories across a whole video sequence.

17.1 The Multi-Frame Tracking Algorithm

The basic algorithm tracks salient points (like corners) through the sequence:

1. **Initialization**: Given a video sequence $I(0), I(1), \ldots, I(T)$, extract a set of corners c_i from the initial frame $I(0)$ for $i = 1, \ldots, N$.
2. **Update Loop**: For each pair of adjacent frames $I(t)$ and $I(t+1)$:
   - For each tracked corner $c_i(t)$ on $I(t)$, estimate its optical flow (using methods like Lucas-Kanade) to find its updated position $c_i(t+1)$.
   - Update the trajectory record for c_i.

Output: A set of N corner tracks detailing the path of each point over time.

Numerical Example: A corner is detected at $c_1(0) = (10, 10)$. Between $t = 0$ and $t = 1$, optical flow estimates $\mathbf{u} = (1, -1)$. New position: $c_1(1) = (11, 9)$. Between $t = 1$ and $t = 2$, optical flow estimates $\mathbf{u} = (2, 0)$. New position: $c_1(2) = (13, 9)$. The track for c_1 is $[(10, 10), (11, 9), (13, 9)]$.

17.2 The Kalman Filter in Tracking

Because optical flow estimates are noisy and sometimes fail (due to occlusion or blurring), we use a **Kalman Filter** to maintain robust tracks.

17.2.1 Key Roles of the Kalman Filter

- **Smoothing**: It smooths out noisy trajectories by blending the actual measurements with a physical prediction model (e.g., constant velocity).
- **Bridging Gaps**: If a feature is temporarily lost (occluded), the filter uses the estimated state (prediction) as a hypothesis to fill in the missing measurement.
- **Uncertainty Tracking**: It maintains a state covariance matrix P_t, which provides a measure of uncertainty, often visualized as an uncertainty ellipse around the predicted position.

17.3 The Data Association Problem

When tracking multiple targets simultaneously, the predictions of where targets will be in the next frame can overlap.

If Track 1 and Track 2 are close to each other, their prediction ellipses might intersect. When a new corner is detected in that intersection area, the system faces the **Data Association Problem**: determining which track the newly detected measurement actually belongs to.

Solutions typically involve calculating distances (like the Mahalanobis distance, which accounts for the shape of the uncertainty ellipse P_t) and assigning the measurement to the most probable track.

18 Principles of 3D computer vision: Introduction

Before diving into complex math to extract 3D data from 2D images (passive vision), it is important to understand **Active 3D Sensors**. These are hardware solutions that actively emit energy into the environment to measure distance:

- **Time-of-Flight (ToF):** These sensors measure the exact time it takes for a light pulse (like a laser) to bounce off an object and return to the photodetector.
- **LIDAR / RADAR / IR:** LIDAR uses lasers for short/medium range, RADAR uses radio frequencies for medium/long range, and IR (Infrared) is often used for short-range depth cameras.

18.1 The Geometry of Image Formation

When using standard cameras, we model how a 3D world gets flattened onto a 2D sensor using the **perspective projection** (or pinhole camera model).

- **The 3D World Point:** We define a point in the real world using 3D coordinates relative to the camera: $P = (X, Y, Z)$.
- **The 2D Image Point:** This 3D point projects onto the camera's image plane (the digital sensor) at a specific 2D pixel coordinate, located at a distance f (the focal length) from the camera's optical center: $p = (x, y, f)$.

The mathematical projection is based on similar triangles:

$$x = f\frac{X}{Z}$$
$$y = f\frac{Y}{Z}$$

Numerical Example:
Imagine your camera has a focal length $f = 50$ mm. You are looking at an object located 1000 mm to the right ($X = 1000$), 500 mm up ($Y = 500$), and 5000 mm away from you in depth ($Z = 5000$). Where does it appear on the camera sensor?

- $x = 50 \cdot (1000/5000) = 50 \cdot 0.2 = 10$ mm
- $y = 50 \cdot (500/5000) = 50 \cdot 0.1 = 5$ mm

The object will be projected onto the sensor 10 mm to the right and 5 mm up from the center.

18.2 The Core Problem: Scale Ambiguity

The fundamental flaw of single-camera 2D vision is **Scale Ambiguity**. Because the perspective projection is a "lossy" and "non-invertible" transformation (meaning depth information Z is lost during the projection), the camera cannot tell the difference without a prior knowledge between a small object up close and a massive object far away.

Revisiting the math example: $X/Z = 1000/5000 = 0.2$.
What if the object was twice as big and twice as far away? ($X = 2000, Z = 10000$). The ratio 2000/10000 is still 0.2. The object projects to the exact same 10 mm spot on the sensor!

18.3 The Solution: A Second View

To recover the lost depth, we add a second camera. By capturing the same 3D point P from two different angles, it will project to point p on the first image plane and p' on the second image plane. Comparing the difference between these two 2D points allows us to compute the original depth.

19 Stereopsis

Stereo vision (or stereopsis) is the process of inferring 3D structure and distance from two or more images taken from different viewpoints. This field of computer vision is heavily inspired by human biology. Because our eyes are set slightly apart, each eye gets a slightly different view of the world. Our brain perceives 3D depth by calculating the difference in the retinal position of the objects we see. This difference is called **disparity**.

19.1 The Math of a Simple Stereo System

Imagine two identical cameras placed side-by-side, perfectly aligned, looking straight ahead.

- f: The focal length of both cameras.
- T: The "baseline," which is the physical physical distance between the optical centers of the two cameras.
- x_l and x_r: The horizontal pixel coordinates where a specific 3D point P appears on the left and right image sensors, respectively.

The difference between these two points is the **disparity**: $d = |x_l - x_r|$. Depth (Z) is calculated using the following formula:

$$Z = \frac{fT}{d} \tag{37}$$

The Golden Rule of Stereo Vision: Depth is *inversely* proportional to disparity.

- Large disparity = The object is very close.
- Small disparity = The object is far away.
- Zero disparity = The object is infinitely far away (like the horizon).

Numerical Example:
Let's say you have a stereo camera rig where the lenses are $T = 100$ mm apart, and both have a focal length of $f = 50$ mm. You take a picture of an object. On the left camera sensor, the object is at pixel coordinate $x_l = -10$ mm. On the right camera sensor, it is at $x_r = -30$ mm.

- Disparity: $d = |-10 - (-30)| = 20$ mm.
- Depth: $Z = \frac{50 \cdot 100}{20} = \frac{5000}{20} = 250$ mm.

19.2 Geometric Derivation of the Depth Equation

Why does $Z = fT/d$ work? It is based on the exact same similar triangles principle from the pinhole camera model, applied twice.

Imagine looking at the camera setup from a top-down view:

1. Let the **left camera's optical center** be at the origin $(0, 0)$.
2. Let the **right camera's optical center** be shifted to the right by the baseline T, so it sits at $(T, 0)$.
3. A 3D point P exists in the real world at coordinates (X, Z).

Left Camera Projection:
Using similar triangles, the light ray from P to the left camera intersects the image plane (at distance f) at coordinate:

$$x_l = f\frac{X}{Z} \tag{38}$$

Right Camera Projection:
Because the right camera is physically moved to the right by T, the relative X coordinate of the point P from the perspective of the right camera is $(X - T)$. Using similar triangles again, the projection on the right image plane is:

$$x_r = f\frac{X - T}{Z} \tag{39}$$

Calculating Disparity:
Now, we define disparity d as the difference between the left and right image coordinates:

$$\begin{aligned} d &= x_l - x_r \\ d &= \left(f\frac{X}{Z}\right) - \left(f\frac{X-T}{Z}\right) \\ d &= \frac{fX - f(X-T)}{Z} \\ d &= \frac{fX - fX + fT}{Z} \\ d &= \frac{fT}{Z} \end{aligned}$$

Solving for Z, we get our final depth equation:

$$Z = \frac{fT}{d} \tag{40}$$

This beautifully shows that the X coordinate (lateral position) cancels out entirely! Depth depends only on the camera properties (f, T) and the measured disparity (d).

19.3 The Two Main Problems of Stereopsis

While the formula is simple, making a computer execute it is difficult. Stereo vision covers two main problems:

1. **The Correspondence Problem:** Given a specific pixel in the left image, how do we find the *exact same* pixel in the right image? Solving this produces dense disparity maps.
2. **The 3D Reconstruction Problem:** Once we have all the disparities, how do we translate them back into the real world to build a full 3D point cloud?

20 Correspondence Problem

To calculate depth using stereo vision, we must first determine the disparity ($d = x_l - x_r$). The **Correspondence Problem** is the challenge of figuring out which pixel in the right image corresponds to a specific pixel in the left image. To solve it, we must decide which image elements to match and which similarity measure to use.

20.1 Sparse vs. Dense Correspondences

We can approach matching in two ways:

- **Sparse Correspondences:** We only try to match highly distinct features, like corners or sharp edges. This is faster but only provides depth for a few specific points.
- **Dense Correspondences:** We attempt to find a match for *every single pixel* in the image to create a full "disparity map". In a standard color-coded disparity map, bright areas represent high disparities (objects closer to the camera), and dark areas represent low disparities (objects farther away).

20.2 Image Rectification

Searching an entire 2D image for a matching pixel is computationally massive. To simplify this, we mathematically warp the stereo pair of images in a process called **Rectification**. Rectification perfectly aligns the two cameras virtually so that their image planes are coplanar.

Because of this geometry, a point on row y in the left image will *always* lie on the exact same row y in the right image (conjugate points lie on corresponding scanlines). Instead of searching a whole 2D image, we only have to search a single 1D horizontal line!

20.3 The Block Matching Algorithm (Dense Correspondences)

To find matches, we cannot simply compare single pixels, as many pixels share the same color/intensity. Instead, we use correlation methods applied to small image patches (a neighborhood or window of pixels).

Algorithm Sketch:

1. Take a small window W (e.g., a 5×5 pixel box) centered on a pixel at coordinates (i, j) in the left image I_l.
2. Slide an identical window along the corresponding row in the right image I_r, shifting it by a disparity distance d. We restrict d to a specific search range $[d_{min}, d_{max}]$.
3. Calculate a similarity score $c(d)$ between the two windows at each step. Common metrics include **SSD** (Sum of Squared Differences) or **NCC** (Normalized Cross-Correlation).
4. The true disparity $\overline{d}$ is the shift that yields the best similarity score (e.g., the minimum difference for SSD).

Numerical Example: Sum of Squared Differences (SSD)
Let's look at a 1D example. We have a small window of 3 pixels in the left image: $[4, 8, 5]$. We are searching along a row in the right image: $[1, 4, 8, 5, 2]$.

- **Test $d = 0$ (shift by 0):** Compare $[4, 8, 5]$ with $[1, 4, 8]$.
  SSD $= (4-1)^2 + (8-4)^2 + (5-8)^2 = 9 + 16 + 9 = 34$
- **Test $d = 1$ (shift by 1):** Compare $[4, 8, 5]$ with $[4, 8, 5]$.
  SSD $= (4-4)^2 + (8-8)^2 + (5-5)^2 = 0 + 0 + 0 = 0$

Since we want the *minimum* difference, $d = 1$ is our winner. The disparity is 1 pixel.

20.4 Left-Right Consistency and Occlusions

Correspondences are made more difficult by **occlusions** (points in the 3D world that are visible to one camera but hidden behind an object from the perspective of the other camera). A point with no counterpart will break the matching algorithm.

To solve this, we enforce **Left-Right Consistency**:

1. Compute a disparity map from the left image to the right image (D_{lr}).
2. Compute a second disparity map from the right image to the left image (D_{rl}).
3. A match is considered valid if the left-to-right disparity is equal and opposite to the right-to-left disparity:

$$D(i, j) = d \iff D_{lr}(i, j) = -D_{rl}(i, j + d) = d \tag{41}$$

If they disagree, we mark that pixel as an occlusion.